



ÍNDIX DE CONTINGUTS

INTRODUCCIÓ

APIREST

APIREST AMB FASTAPI

Entorn virtual

Instal·lació FastAPI

Endpoints

SQLModel

Variables d'entorn

requirements.txt

CORS

BaseModel

INTRODUCCIÓ

FastAPI és un **framework** de **python** orientat a **servidor**. Un framework és un conjunt d'eines, biblioteques i convencions que ens proporciona una estructura per desenvolupar aplicacions de manera més ràpida i consistent.

En lloc de construir tot des de zero, un framework ens ofereix:

- Estructura organitzada per al nostre codi
- Funcionalitats preconstruïdes per a tasques comunes
- Millores en productivitat i qualitat del codi
- Patrons de disseny provats i estandarditzats

Principals frameworks de Python:

FastAPI	Creació de APIRest de forma senzilla i ràpida
Flask	Lleuger tot i que requereix més configuració manual.
Django	Complet, estructurat (MVT), ideal per aplicacions grans amb interfície web
Pyramid	Flexible i modular

Patrons de disseny i Organització

Normalment, els frameworks segueixen algun patró de disseny com el **MVC** (**Model View Controller**) el qual treballarem amb **Django**.

FastAPI, en canvi, tot i que també pot seguir patrons semblants, està orientat més a la creació d' **APIRest** de forma **ràpida**, **potent** i **senzilla**. Tot i que cal tenir un coneixement d'aquests patrons molt avançat per crear projectes amb FastAPI, l'utilitzarem per entendre els conceptes d'APIRest de forma **ràpida** i **senzilla**.

Punts forts de **FastAPI**:

- Ràpid: en execució i desenvolupament.
- Senzill: poc codi per crear endpoints.
- Potent: soporta programació [asíncrona](#) i síncrona.
- Documentació automàtica: genera documentació d'endpoints amb Swagger

Punts febles de **FastAPI**:

- Coneixements previs patrons de disseny.
- Cal seguir alguna estructura.
- Configuració manual.

Comunicació d'un APIRest amb FastAPI

Un **APIRest** respon a les consultes dels clients i es comunica de manera nativa mitjançant JSON. Aquest format permet comunicar-se amb qualsevol tecnologia capaç de llegir/enviar JSON (web en Javascript, mòbils, etc.)

- Amb **FastAPI**, la comunicació amb el client es fa amb JSON per defecte.
- Retornar un diccionari de Python fa que FastAPI el converteixi automàticament en JSON abans d'enviar la resposta al client **utilitzant les eines adequades**.

Eines més importants que utilitza FastAPI

- [Starlette](#):

És un framework lleuger i flexible que utilitza FastAPI per a construir serveis asíncrons basats en el protocol ASGI (Asynchronous Server Gateway Interface).

- Permet construir serveis web HTTP i WebSockets.
- Suport per a tasques en segon pla dintre del procés.
- Inclou un client de proves amb httpx.
- Suport per a CORS (comunicació amb el client).
- Permet funcionar amb diferents backends asíncrons.

- [Pydantic](#):

És una biblioteca de validació de dades per Python. És flexible i ràpida.

- Utilitza les declaracions de tipus Python per definir esquemes de validació i serialització de dades. Implica menys codi i millor integració amb l'IDE.
- La lògica de validació central de Pydantic està escrita en Rust. Implica molta velocitat en validació de dades per Python.
- Suport per a JSON Schema facilitant la integració amb altres eines.
- Integració amb altres biblioteques com SQLAlchemy

- [Uvicorn](#):

És una eina essencial per executar aplicacions web asíncrones. S'encarrega de gestionar la comunicació de la xarxa rebent sol·licituds de clients i enviant resposta. És compatible amb el protocol ASGI. Pot ser utilitzat en entorn de desenvolupament com producció.

- [SQLModel](#):

És una llibreria per interactuar amb bases de dades SQL amb Python. És molt intuïtiu i d'ús senzill i molt robust. Combina SQLAlchemy (ORM de Python) amb Pydantic per a simplificar el codi.

- [SQLAlchemy](#)→:

És una llibreria de Python que facilita la interacció amb bases de dades SQL. És un ORM que permet treballar amb taules i registres com si fossin objectes de Python.

APIREST



Una APIREST (Representational State Transfer) és un conjunt d'interfícies de programació d'aplicacions (API) que permet que diferents clients (web, mòbil, rellotges, IoT...) es puguin comunicar en un mateix servidor i compartir dades mitjançant el protocol **HTTP**.

PRINCIPIS DE L'ARQUITECTURA REST

- **Sistema client-servidor:** REST separa el client i el servidor, permetent-los evolucionar independentment. Això permet una millor escalabilitat i mantenibilitat del sistema.
- **Interfície uniforme:** Les operacions a través de l'API es realitzen de manera uniforme utilitzant mètodes HTTP estàndard, com GET, POST, PUT, PATCH i DELETE. Cada recurs té una URL única que l'identifica.
- **Stateless:** Les dades del sistema es transfereixen entre el client i el servidor en forma de representacions d'estat del recurs. Això significa que cada sol·licitud al servidor inclou tota la informació necessària per processar-la, sense dependre de cap estat de sessió emmagatzemat al servidor.

ELEMENTS BÀSICS QUE CONFORMEN UNA API RESTFUL

- **Recursos (Resources)**
Entitats que es poden consultar o manipular a través de l'API (usuaris, productes, comandes)
Exemples: /usuaris, /productes, /comandes

- **Mètodes HTTP**

- **GET** Recuperar recursos
- **POST** Crear nous recursos
- **PUT** Actualitzar/reemplaçar recursos
- **PATCH** Actualització parcial
- **DELETE** Eliminar recursos

- **Endpoints/URLs**

URLs que identifiquen recursos específics. Segueix una estructura jeràrquica i lògica.

Els endpoints poden dir-se igual sempre que el mètode sigui diferent. És a dir, no hi pot haver dos endpoints que es diguin igual i siguin d'un mateix mètode

Ex:

GET	/api/v1/usuaris, /api/v1/usuari,
POST	/api/v1/usuaris
GET	/api/v1/usuaris/123
PUT	/api/v1/usuaris/123
DELETE	/api/v1/usuaris/123

- **Codis d'Estat HTTP**

Cada resposta d'una petició ha d'anar sempre acompanyada d'un status que indica l'estat de la petició.

2xx - Èxit:

- 200 OK - Petició exitosa
- 201 Created - Recurs creat

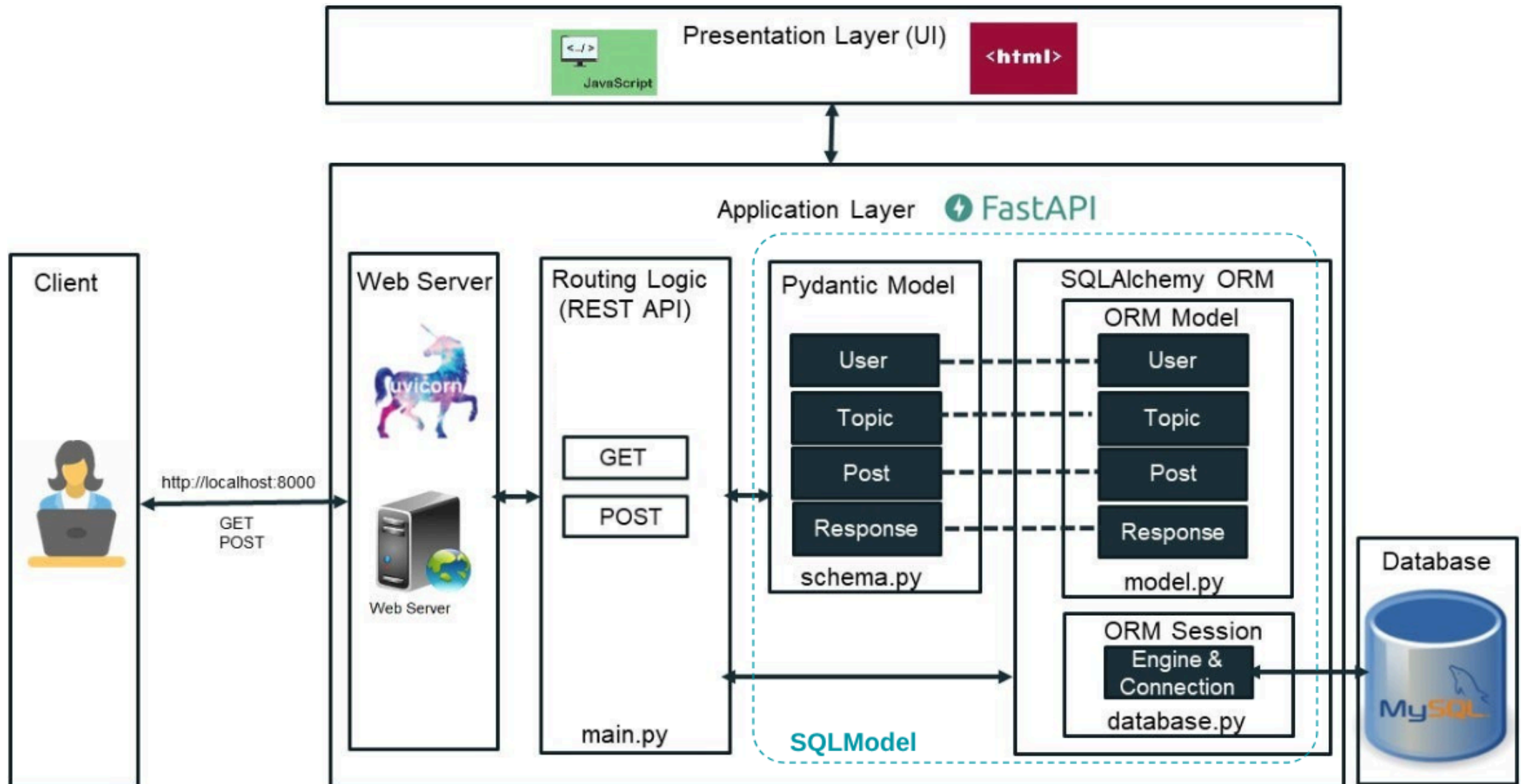
4xx - Error del client:

- 400 Bad Request - Petició mal formada
- 401 Unauthorized - No autenticat
- 403 Forbidden - No autoritzat
- 404 Not Found - Recurs no trobat

5xx - Error del servidor:

- 500 Internal Server Error - Error genèric
- 503 Service Unavailable - Servei no disponible

API REST AMB FASTAPI



Entorn virtual

Abans d'instal·lar FastAPI es **recomana** la creació d'un entorn de treball virtual amb **virtualenv**.

Virtualenv és una eina per aïllar diferents entorns amb diferents versions de Python. **NO** és una Màquina virtual, però ens serveix per virtualitzar l'entorn segons les versions de les tecnologies necessàries per a realitzar el projecte.

Seguint la [documentació oficial](#) de virtualenv s'indica les passes a seguir per instal·lar **virtualenv** i com crear un entorn virtual (**venv**).

Comanda per instal·lar **virtualenv**:

```
python -m pip install virtualenv    o    python3 -m pip install virtualenv
```

CREACIÓ ENTORN

Una vegada instal·lat **virtualenv** es pot crear tants entorns virtuals com es vulgui amb la següent comanda:

```
python3 -m venv <.env_nom>
```

ACTIVACIÓ ENTORN VIRTUAL ➔

Es recomana activar l'entorn cada vegada que es treballi amb el projecte. Si no es fa així, pot ocasionar errors a la llarga.

Per activar en **Linux** l'entorn creat: `source .env_nom/bin/activate`

Per desactivar l'entorn virtual actiu: `deactivate`

Per altres Sistemes Operatius, a l'enllaç teniu les comandes.

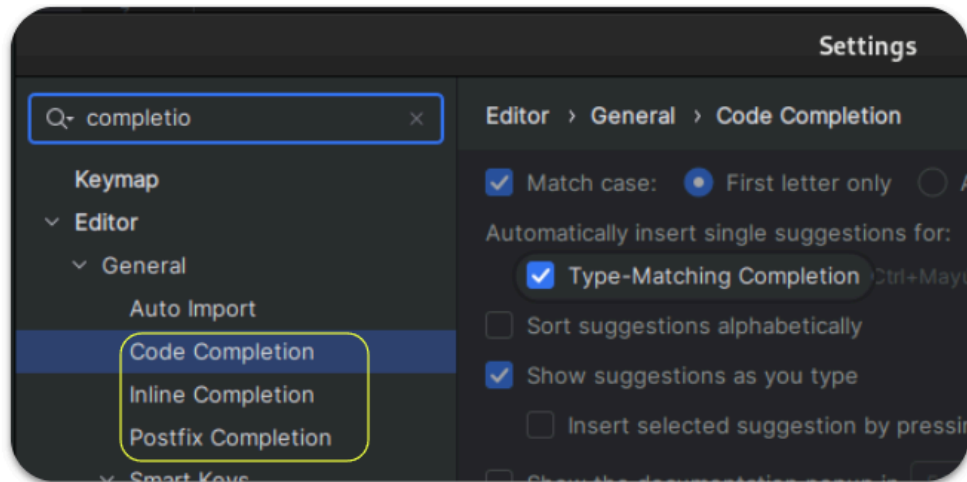
Un cop activat l'entorn virtual es pot començar amb la instal·lació

Comanda per saber els paquets instal·lats dintre d'un **venv** `pip list`

NO ÉS OBLIGATORI PERÒ SI MOLT RECOMANABLE TREBALLAR AMB VIRTUALENV EN PROJECTES DE PYTHON.

Configuració PyCharm (obligatori)

És obligatori desactivar totes les opcions de **code completion**, **postfix completion** i **inline completion**. Vigileu ja que (degut a un bug) pot passar que ho hagueu de fer cada vegada que entreu a l'editor.



Instal·lació FastAPI

És obligatori seguir aquesta documentació i, en el cas de **FASTAPI**, seguir la [documentació oficial](#) ja que està amb molts exemples i molt ben explicada.

És important seguir les explicacions i demandes del professorat ja que a la documentació oficial hi ha conceptes molt avançats i mètodes molt complicats que **NO** es demanen per avaluar. La documentació oficial és com a complement per a l'aprenentatge.

Per instal·lar fastapi caldrà posar la següent comanda

```
python3.10 -m pip install "fastapi[standard]" o pip install "fastapi[standard]"
```

Un cop instal·lat ja es pot començar a treballar en el projecte.

Endpoints

Un endpoint és un direccionament URL on s'indica la consulta del client i s'hi prepara una resposta. Cal crear un arxiu **main.py** a l'arrel del projecte i afegir el següent codi:

main.py

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/api/hello_str")
async def greetings():
    return "hello"
```

El decorador **@app.get()** associa una funció (asíncrona o síncrona) amb una ruta específica i un mètode HTTP (GET, POST, PUT, DELETE).

app = FastAPI() és una instància de la classe **FastAPI()** que proporciona el framework fastapi. Aquesta instància té tota la configuració i l'estat de l'aplicació. FastAPI utilitza la instància per generar automàticament la documentació interactiva en format Swagger amb la URL **localhost:8000/docs**.

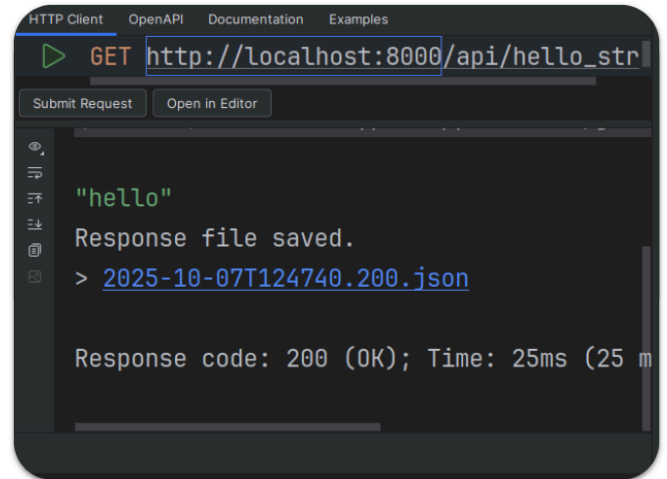
@app.get("/api/hello_str", response_model=str) és el decorador que associa una ruta arrel ("**/api/hello_str**") a una funció utilitzant el mètode GET de HTTP.

response_model indica quina és la resposta que es vol enviar a través del return de la funció asíncrona (**async def**).

El decorador realitza el següent:

- **Registra la ruta** li diu a FastAPI que la ruta "**/api/hello_str**" ha de ser gestionada per la funció asíncrona **greetings()**.
- **Especifica al mètode HTTP** que la ruta només donarà resposta a sol·licituds GET.

- **Processa la sol·licitud i la resposta:**
FastAPI automàticament s'encarrega d'interpretar els paràmetres de la sol·licitud, validar les dades, generar la documentació automàtica (OpenAPI) i convertir la resposta en JSON.



async def greetings(): funció asincrònica que gestiona la sol·licitud.

Una funció **sincrònica** executa el seu codi de forma seqüencial (línia a línia). Si troba una operació (com una consulta a una Base de Dades) para l'execució fins que aquesta operació sigui completada.

Una funció **asincrònica** executa el seu codi de forma seqüencial (línia a línia). Si troba una operació (com una consulta a una Base de Dades) no para l'execució per a poder gestionar altres operacions.

return resposta de la funció segons l'indicat a **response_model**.

MÈTODES HTTP →

Els mètodes que s'utilitza amb HTTP són **GET**, **POST**, **PUT**, **DELETE** i d'altres (com Option, Patch, etc)

- **GET**

El mètode GET del protocol HTTP s'encarrega de les sol·licituds del client per recuperar informació de la Base de Dades.

main.py

```
@app.get(path: "/api/hello_dict", response_model=dict)
def greetings_dic():
    return {"msg": "hello"}
```

- **POST**

El mètode POST del protocol HTTP s'encarrega de les sol·licituds del client per inserir informació nova a la Base de Dades.

main.py

```
@app.post(path: "/api/insert_hello", response_model=dict)
def insert_greeting():
    return {"msg": "Inserit missatge de benvinguda"}
```

- **PUT**

El mètode PUT del protocol HTTP s'encarrega de les sol·licituds del client per modificar informació a la Base de Dades.

main.py

```
@app.put(path: "/api/update", response_model=dict)
async def update_greeting():
    return{"msg":"update successfully"}
```

- **DELETE**

El mètode DELETE del protocol HTTP s'encarrega de les sol·licituds del client per eliminar informació de la Base de Dades.

main.py

```
@app.delete(path: "/api/delete", response_model=dict)
async def delete_greeting():
    return{"msg":"deleted successfully"}
```

Execució projecte:

fastapi dev main.py

o

uvicorn main:app --reload

SQLModel (per crear taules)

SQLModel és una llibreria per ajudar amb les consultes SQL a les bases de dades. Aquesta llibreria ajuda a escriure objectes en Python i transferir aquesta informació a una Base de Dades a través de consultes SQL.

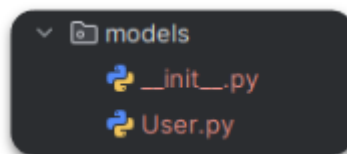
Abans de poder fer cap consulta (CRUD) a la Base de Dades (**BD** o **DB**) caldrà definir les taules a crear a la BD.

Normalment, el que es fa és crear les taules a través de llenguatge SQL (**CREATE TABLE,...**), un cop creades les taules ja es podria començar a fer aquestes consultes.

Amb la llibreria **SQLModel** es pot automatitzar aquesta creació de taules sense utilitzar consultes SQL creant classes que utilitzen la llibreria SQLModel per vincular-les a la BD.

PASSES PER CREAR UNA TAULA A LA BD

1. Crear una carpeta de nom **models** al directori principal del projecte.
2. Dintre de la carpeta afegir un fitxer de nom **__init__.py** que estarà completament buit i un altre fitxer amb el nom de la taula a crear, per exemple **User.py**.



3. Crear una classe de nom **User** dintre del fitxer **User.py** i en aquesta classe caldrà afegir-hi els camps i el tipus de dada de cada camp.

User.py

```
from sqlmodel import SQLModel, Field

class User(SQLModel, table=True):
    id: int = Field(default=None, primary_key=True)
    name: str
    email: str
```

table = True s'utilitza per indicar a **SQLModel** que aquesta classe representa una taula

id serà una clau primària (**primary key=True**) i no serà obligatori que estigui afegit a la consulta (**default=None**). Per indicar aquesta configuració cal utilitzar el mètode **Field**.

name i **email** es defineixen com a tipus de dades string (**str**)

4. **Afegir** el següent codi a **main.py**

- Carregar **variables d'entorn** → amb **load_dotenv()**. Caldrà assegurar-se que està l'import **from dotenv import load_dotenv**
- Configurar la connexió a la BD (en l'exemple a PostgreSQL)
- Creació de les taules a la BD

main.py

```
from sqlalchemy import SQLModel, create_engine, Session
from dotenv import load_dotenv
import os

#a
load_dotenv()
#b
DATABASE_URL = os.getenv("DATABASE_URL")
engine = create_engine(DATABASE_URL)
#c
SQLModel.metadata.create_all(engine)
```

Amb **DATABASE_URL = os.getenv("DATABASE_URL")** es guarda a la variable **DATABASE_URL** la variable d'entorn ubicada a l'arxiu **.env**. Per poder agafar aquesta variable d'entorn s'utilitza el mètode **getenv** de la llibreria **os**.

Amb **engine = create_engine(DATABASE_URL)** es crea un objecte de connexió a la BD de nom **engine**. Amb el mètode **create_engine** s'agafa la variable d'entorn que conté totes les dades de connexió i realitza aquesta connexió creant un objecte de connexió i guardant-la a la variable **engine**.

Amb **SQLModel.metadata.create_all(engine)** es creen les taules a la BD. Amb el mètode **metadata.create_all** es passa per paràmetre l'objecte de connexió **engine** i es crea la migració dels models creats amb **SQLModel** creant així les taules a la BD de forma automàtica.

Per poder connectar amb la BD i poder dur a terme les accions del CRUD cal crear les sessions.

main.py

```
def get_db():
    db = Session(engine)
    try:
        yield db
    finally:
        db.close()
```

db = Session(engine):

Aquí es crea una nova sessió de base de dades utilitzant un objecte **engine** (que conté la configuració de connexió a la BD)

La sessió (**db**) és l'objecte que permet interactuar amb la BD (fer consultes, afegir dades, etc.)

yield db:

yield retorna la sessió de la BD i es pot utilitzar en tot l'entorn del projecte.

Permet que la sessió estigui disponible mentre s'executa una petició HTTP (per exemple, quan es crida un endpoint).

finally: db.close()

En aquesta línia de codi s'executa sempre, tant si la petició es completa correctament com si hi ha un error.

Aquí es tanca la sessió de la BD per alliberar recursos i evitar problemes de connexió

A continuació es mostra un exemple d'ús de **get_db** en un endpoint:

Depends(get_db)

Depends és una eina de FastAPI que s'utilitza per injectar dependències en els endpoints.

En aquest cas, injecta la sessió de la BD (**db**) que retorna la funció **get_db**.

db:Session

Aquest és el paràmetre que rep l'endpoint. Conté la sessió que es crea amb **get_db()**.

Gràcies a **Depends**, FastAPI s'encarrega de cridar **get_db()**, obtenir la sessió (**db**) i passar-la a l'endpoint.

Aquesta connexió a la **db** és important degut a:

- **Gestió de recursos:** la funció **get_db()** assegura que la connexió a la BD es tanqui correctament després de cada petició, evitant problemes com connexions obertes innecessàriament.
- **Injecció de dependències:** **Depends(get_db)** simplifica el codi de l'endpoint, ja que FastAPI s'encarrega de gestionar la sessió de BD automàticament.
- **Reutilització:** la funció **get_db()** es pot utilitzar en múltiples endpoints, evitant la duplicat de codi

Variables d'entorn

Per a carregar les variables d'entorn caldrà crear un fitxer de nom **.env** en el que s'hi allotjarà la url de la BD per a crear la connexió al moment d'executar el codi.

Aquest arxiu es crea al directori principal del projecte amb el següent codi:

```
.env
```

```
DATABASE_URL=postgresql+psycopg2://usuari:contrasenya@localhost:5432/nom_db
```

DATABASE_URL és el nom de la variable d'entorn que es podrà utilitzar en tots els arxius del projecte.

postgresql+psycopg2 indica la Base de Dades que s'utilitzarà i la llibreria de python associada a postgresql.

usuari:contrasenya dependrà de la Base de Dades creada. Si s'ha utilitzat un fitxer **.YML** caldrà veure el l'apartat **environment** del servei de la BD i buscar-hi les dades corresponents a l'usuari i la contrasenya. En cas d'haver utilitzat un executable de postgresql caldrà afegir-hi les dades segons la BD creada amb la shell.

nom_db correspon al nom de la BD creada amb anterioritat, ja sigui a través de d'executable o de **.YML**.

Document per a treballar amb [docker](#)

requirements.txt

Aquest fitxer de nom **requirements.txt** conté totes aquelles dependències necessàries pel bon funcionament del projecte. Només s'hi poden posar llibreries que es puguin instal·lar amb el paquet d'instal·lació **pip**.

requirements.txt

```
fastapi
sqlmodel
psycopg2-binary #o psycopg2
python-dotenv
uvicorn
```

Aquest fitxer està ubicat al directori principal del projecte i caldrà posar la següent comanda (en cas de que calgui fer la instal·lació de les dependències). Només cal fer-ho un cop.

terminal

```
pip install -r requirements.txt
# en cas de donar error, provar la següent comanda
python3 -m pip install -r requirements.txt
```

CORS

Tots els ApiREST tenen una part de seguretat contra atacs externs per a que no tothom pugui fer les consultes a la Base de Dades. En aquest projecte, FastAPI només deixa fer consultes des del swagger o altra tecnologia de Python.

Com que el nostre frontend és amb HTML, css i JS, quan es vulgui fer una consulta donarà error de **CORS**. Per evitar aquest missatge caldrà afegir, després de **app = FASTAPI()** al fitxer **main.py** el següent codi:

```
app = FastAPI()

'''
Afegim el CORS per a que la tecnologia que s'utilitzi al
frontend pugui fer consultes als endpoints de la APIREST
'''

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# 1. Carregar variables d'entorn
load_dotenv()
```

Connexió BD

En aquest apartat es mostrarà com crear 2 serveis amb **docker** en un sistema **Debian** [→](#). Per a fer les activitats **NO** és obligatori utilitzar docker per crear els serveix. És una eina més a aprendre a utilitzar.

Instal·lació docker

A **windows** cal instal·lar-se **docker desktop** i cal seguir les passes indicades a la instal·lació de la documentació oficial [→](#).

Amb **Debian** o qualsevol altra versió de linux cal instal·lar **docker engine** [→](#).

Creació arxiu .yaml

Per a poder crear els serveis amb docker cal crear un arxiu de nom **docker-compose.yml** a qualsevol ubicació del PC.

En aquesta imatge es mostra la creació de 2 serveis: **postgresql** (BBDD) i **pgadmin** (gestió de BBDD visual)

```
version: '3.1'
services:
  db:
    image: postgres:latest
    container_name: db_fastapi
    environment:
      - POSTGRES_DB=fastapi_db
      - POSTGRES_PASSWORD=passwd
      - POSTGRES_USER=usuario
    ports:
      - "5432:5432"
    volumes:
      - local_pgdata:/var/lib/postgresql/data
  pgadmin:
    image: dpage/pgadmin4
    container_name: pg_fastapi
    ports:
      - "80:80"
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@admin.com
      PGADMIN_DEFAULT_PASSWORD: admin
    volumes:
      - pgadmin-data:/var/lib/pgadmin
volumes:
  local_pgdata:
  pgadmin-data:
```

Execució arxiu .yml

Per a poder crear els serveis cal obrir una terminal o la terminal de l'IDE, ubicar-se a la mateixa carpeta a on estigui l'arxiu **docker-compose.yml** i posar la següent comanda:

sudo docker compose up -d (windows, la mateixa comanda però sense sudo)

Si és la primera vegada que es crea els serveis d'aquell arxiu es veurà a la terminal uns pulls i trigarà uns minuts, en canvi, si ja s'ha creat el servei amb anterioritat la creació dels serveis serà immediata.

Comandes varies per gestionar docker

Les següent comandes ens permet oferir solucions ràpides a errors típics.

- Comanda per a veure contenidors actius:
sudo docker ps
- Comanda per a veure contenidors actius i inactius:
sudo docker ps -a
- Parar contenidors:
sudo docker stop <container ID>
- Eliminar contenidors:
sudo docker rm <container ID>
- Veure volums creats:
sudo docker volume ls
- Eliminar volums:
sudo docker volume rm <ID>

Comandes en linux per a solució de PORTS

Normalment, si tenim el port que s'està utilitzant per a algun altre servei, es pot modificar el port en l'arxiu **docker-compose.yml** o es pot eliminar el servei del port per a lliurar el port. Aquí es mostra com eliminar els serveis que estan ocupant aquell port que es volen utilitzar.

Amb la següent comanda es demana informació dels serveis al port **80**:

sudo lsof -i:80

Amb la següent comanda s'elimina el servei que ocupa el port **80**:

sudo kill <PID> o sudo kill -9 <PID>

Utilització PGADMIN

Si es fa servir l'arxiu **docker-compose.yml**, aquí es mostra com accedir a la BD creada amb el **.yml** en el servei de **pgadmin**.

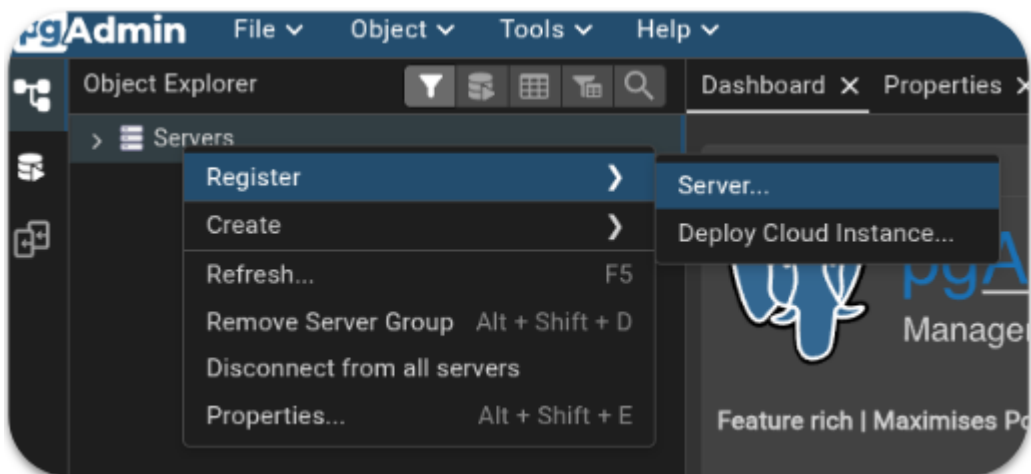
Un cop que els serveis del **.yml** ja estan actius, només cal anar a qualsevol explorador i posar com url: **localhost:80**.

Això s'indica en la secció de ports del servei de **pgadmin** de l'arxiu **.yml**

En l'explorador es mostrarà la pàgina de login del **pgadmin**.
Cal posar les dades indicades a **environment** del **pgadmin**.

```
pgadmin:
  image: dpage/pgadmin4
  container_name: pg_fastapi
  ports:
    - "80:80"
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@admin.com
    PGADMIN_DEFAULT_PASSWORD: admin
  volumes:
    - pgadmin-data:/var/lib/pgadmin
```

Un cop obert el **pgadmin** es mostrarà la pàgina d'inici i per a veure la taula caldrà crear un servei (**Servers > Register > Server...**) amb les dades indicades al **.yml**.



S'obrirà una finestra on caldrà posar les dades indicades al **.yml**. A la pestanya de **General** posar un **Name** que es vulgui.

The screenshot shows the 'Register - Server' dialog box with the 'General' tab selected. The fields are as follows:

Field	Value
Name	nomQueVulgueu
Server group	Servers
Background	☐
Foreground	☐
Connect now?	☒
Shared?	☐

Caldrà obrir la pestanya **Connection** i posar les dades de connexió segons l'arxiu **.yml**.

The screenshot shows the 'Register - Server' dialog box with the 'Connection' tab selected. The fields are as follows:

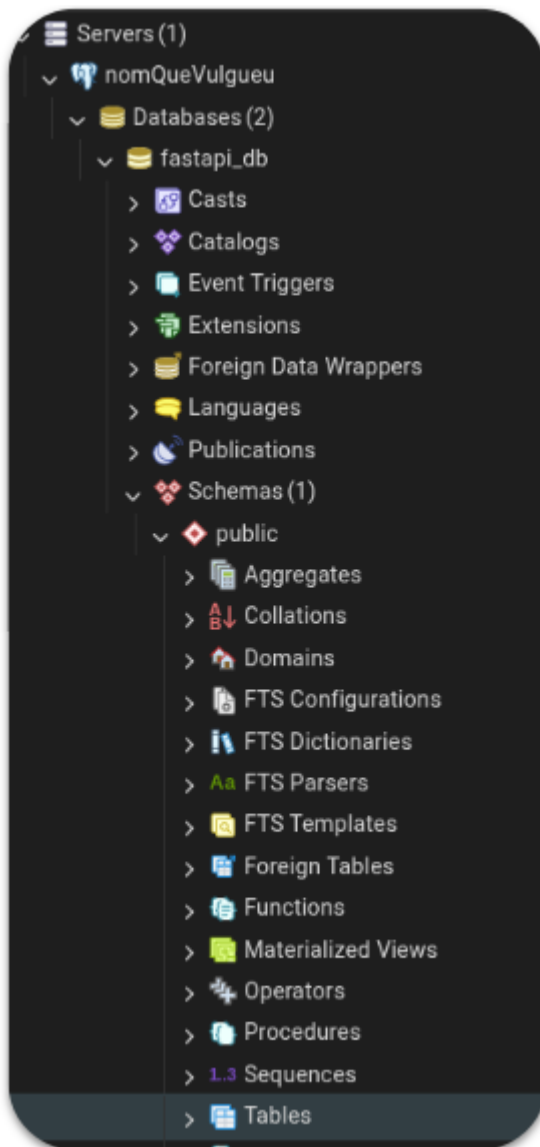
Field	Value
Host name/address	db_fastapi
Port	5432
Maintenance database	postgres
Username	usuario
Kerberos authentication?	☐
Password	
Save password?	☐
Role	

Below the password field, there is a note: "In edit mode the password is true."

To the right of the dialog box, there is a snippet of a **.yml** file with yellow boxes highlighting the values that correspond to the fields in the dialog box:

```
db:
  image: postgres:13
  container_name: db_fastapi
  environment:
    - POSTGRES_DB=fastapi_db
    - POSTGRES_PASSWORD=passwd
    - POSTGRES_USER=usuario
  ports:
    - "5432:5432"
```

Si li doneu a **save** us obrirà una pantalla amb informació varia. Heu d'anar al desplegable de l'esquerra i anar donant a les fletxes > fins a trobar **Tables**. En aquesta secció és on es crearan les taules del projecte.



SQLModel (per a fer CRUD)

Abans d'entrar en detall de com utilitzar SQLModel per a fer consultes, cal que s'entengui l'ús de la classe **BaseModel** de **Pydantic** que podeu veure que està molt extesa i que no és incorrecte.

Utilització de BaseModel

BaseModel → és una classe de **Pydantic** que utilitza **FastAPI** → per a la serialització i verificació de les dades de la request i del response. Permet:

- llegir el cos de la request com a JSON.
- convertir el tipus de dades si cal.
- validar que les dades compleixin els requisits específics.
- generar errors detallats si les dades són incorrectes.
- verificar que les dades de resposta estiguin ben estructurades.

Cal importar la classe **BaseModel** de **Pydantic**. S'ha de crear la classe segons les dades que es vulguin treballar:

```
main.py x
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3
4 app = FastAPI()
5
6 class User(BaseModel): 2 usages
7     name: str
8     lastName: str
9     age: int
10
11 @app.post(path: "/user", response_model=User)
12 def addName(user: User):
13     return user
```



Pydantic, a través de **BaseModel**, s'encarrega de treballar les dades en format json. Aquest tipus de classe es poden fer servir per les dades que envia el client i les dades que **FastAPI** envia al client. S'ha de tenir cuidado amb les dades sensibles com el password, les quals no poden tornar al client.

Si executeu aquest projecte mostrarà una resposta al swagger com a la imatge anterior.

Aquí **NO** estem connectant amb la **BD**.

Problemes amb l'ús de BaseModel

El problema amb BaseModel és que, al voler fer les consultes amb una BD cal transformar les dades de BaseModel en dades SQLAlchemy i s'utilitza un mètode **deprecated** (`.dict()`). Això fa que hi hagi problemes a la llarga i que el codi no sigui tan net i senzill d'entendre.

En resum, amb **BaseModel** es crea una transformació intermitja (que es pot utilitzar) que obliga a crear unes passes extres per a transformació de dades.

És cosa del projecte i del programador utilitzar o no **BaseModel**.

Exemple de endpoint amb **BaseModel**:

```
@app.post(path: "/users", response_model=User)
def addName(user: User, db: Session = Depends(get_db)):
    db_user = User(**user.dict())
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return User(**db_user.dict())
```

Aquest tipus de codi **deprecated** pot arribar a donar errors. Per tant, cal utilitzar mètodes més actualitzats i nets.

Quan utilitzar BaseModel ➡

És important (obligatori) utilitzar **BaseModel** quan en un endpoint li arriba informació d'un formulari en HTML pur i no en format JSON API.

Creació diferents classes amb SQLAlchemy

A la documentació oficial podreu veure un exemple amb la classe **Hero** [→](#). Es poden crear varis models de la mateixa classe depenent de com es vulgui gestionar les dades. Només pot haver-hi una classe amb **table=True** ja que només es pot crear una taula d'aquella classe.

Com a exemple s'han creat 3 classes (1 taula i 2 models)

```
from sqlalchemy import SQLAlchemy, Field

class User(SQLAlchemy, table=True): 2 usages
    id: int | None = Field(default=None, primary_key=True)
    name: str
    lastname: str
    age: int
    passwd: str

class UserRequest(SQLAlchemy): 2 usages
    name: str
    lastname: str
    age: int
    passwd: str

class UserResponse(SQLAlchemy): 1 usage
    id: int
    name: str
    lastname: str
    age: int
```

Creació endpoints per consultar a BD

S'aprofita els serveis creats amb el **docker-compose.yml** per a provar les consultes dels endpoints de **FastAPI** i verificar el **CRUD** a la **BD**.

CREATE

```
<< /user
@app.post(path: "/user", response_model=dict, tags=["CREATE"])
def addUser(user: UserRequest, db: Session = Depends(get_db)):
    # Convertir les dades del client a un model compatible amb la taula User
    insert_user = User.model_validate(user)
    db.add(insert_user)
    db.commit()
    return {"msg": "Afegit usuari correctament"}
```

model_validate és una classe de sqlmodel que ens permet transformar les dades de **user** (objecte json) en dades sqlmodel.

db.add(): **db** és la sessió oberta de la base de dades (Session) i el mètode **add()** ens permet afegir un objecte a la sessió de la base de dades per a una futura inserció.

db.commit() és el mètode que s'encarrega de fer efectius els canvis a la **BD**.

READ by ID

```
<< /user/{id}
@app.get(path: "/user/{id}", response_model=UserResponse, tags=["READ by ID"])
def getUser(id: int, db: Session = Depends(get_db)):
    # Amb els statments de sqlalchemy es crea la consulta i es guarda a stmt
    stmt = select(User).where(User.id == id)
    result = db.exec(stmt).first()
    return UserResponse.model_validate(result)
```

select(User).where(User.id == id) ➡ és la query adaptada al format **sqlalchemy** per a treballar amb taules d'**sqlmodel**.

first() mètode per retornar el primer resultat de la consulta i retorna **none** si no hi ha cap resultat

PROVAR DE MODIFICAR USERRESPONSE PER A VEURE EL CANVI DE RESPOSTA EN EL SWAGGER.